

# DESARROLLO DE INTERFACES

CASO PRACTICO I UD8



ALUMNO CESUR

25/26

Alejandro Muñoz de la Sierra

PROFESOR

Manuel Gómez Lora

# INTRODUCCION

A la hora de plantearnos el desarrollo de este sistema de control de asistencia corporativo (GestorAsistenciaQA), partimos de una premisa innegociable: la calidad del software no podía ser una simple "tiritita" que se pone al final para cumplir con la práctica, sino el pilar central sobre el que construiríamos todo el código. En vez de caer en el error de siempre de liarnos a picar la interfaz gráfica de OpenJFX a toda prisa y luego intentar enchufarle la base de datos a la fuerza, decidimos apostar por una arquitectura de responsabilidades bien separadas.

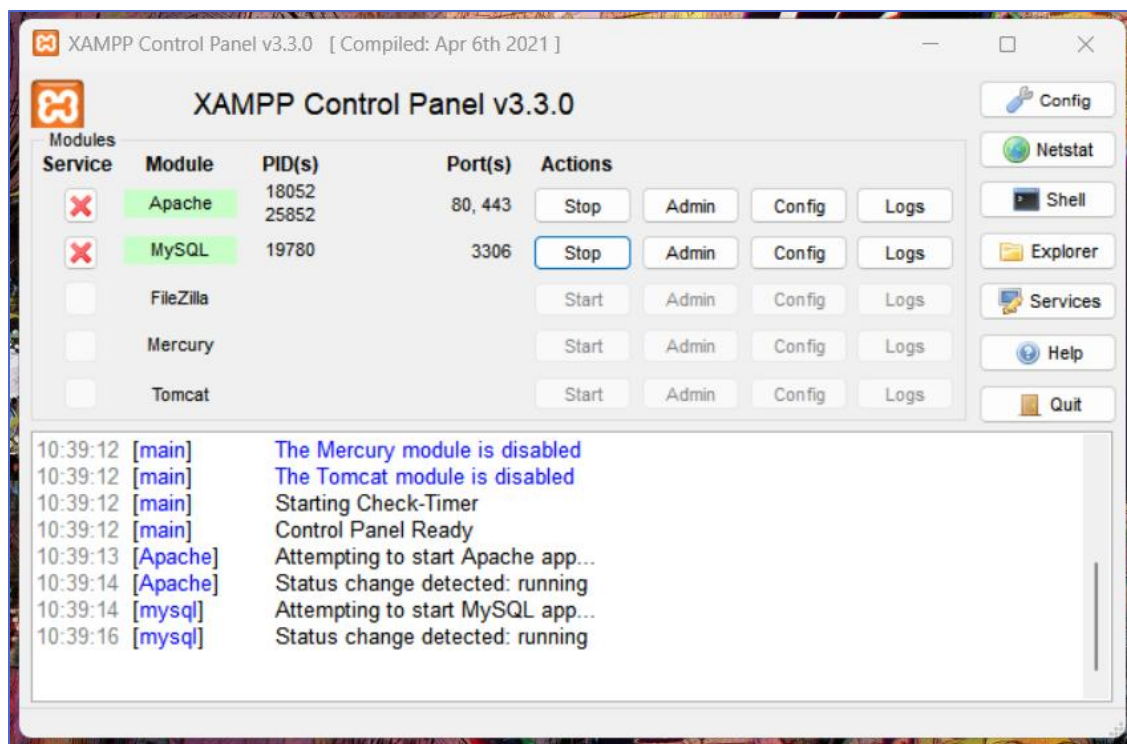
Para lograrlo, diseñamos un componente totalmente aislado para gestionar los datos en MariaDB (usando el entorno local de XAMPP) con un objetivo estratégico muy claro: poder pasarle auditorías automatizadas antes siquiera de pintar un solo botón en la pantalla. Apoyándonos en una estructura limpia con Maven y aprovechando la compatibilidad de Java 21 LTS, nuestra idea desde el primer minuto ha sido asimilar un flujo real de validación y verificación de software.

Queríamos garantizar no solo que el programa haga lo que pide el cliente, sino que sea robusto frente a fallos y, sobre todo, resista a manipulaciones de seguridad maliciosas.

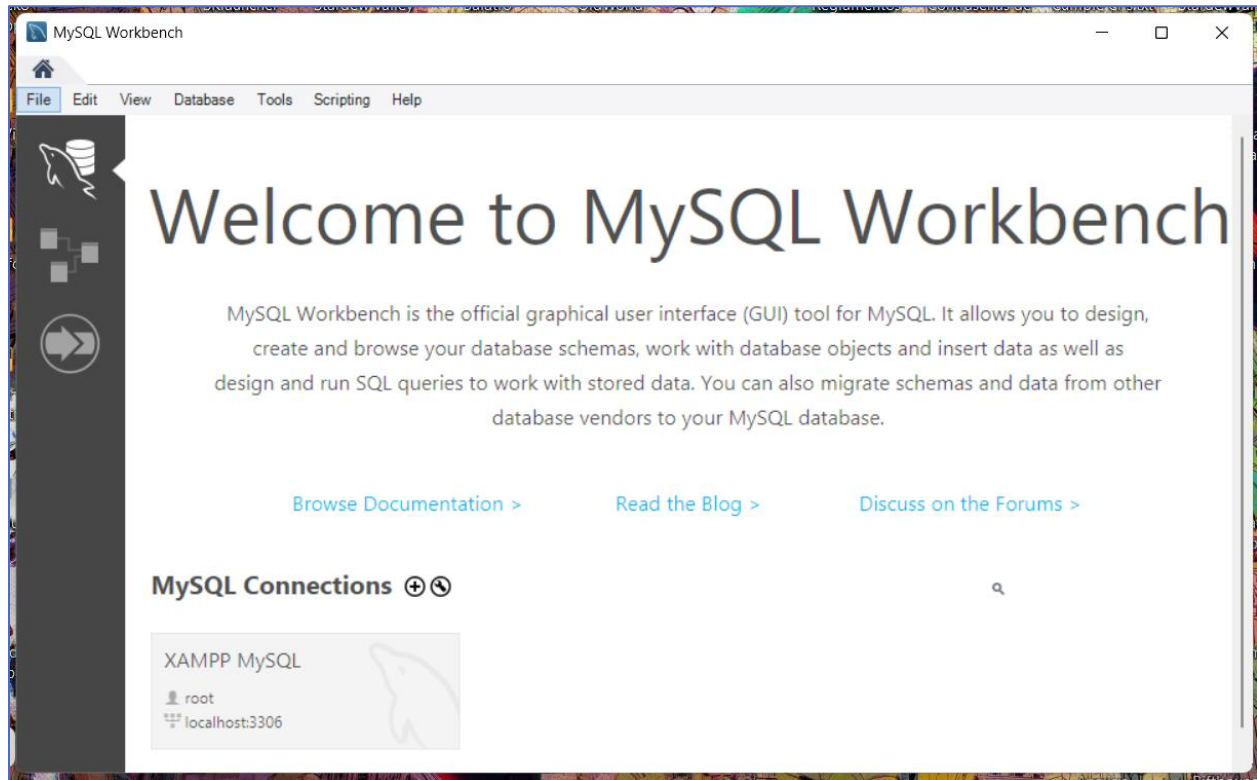
0 1

# ESTABLECIMIENTO DEL ENTORNO DE DESARROLLO E INTEGRACIÓN DE LIBRERÍAS

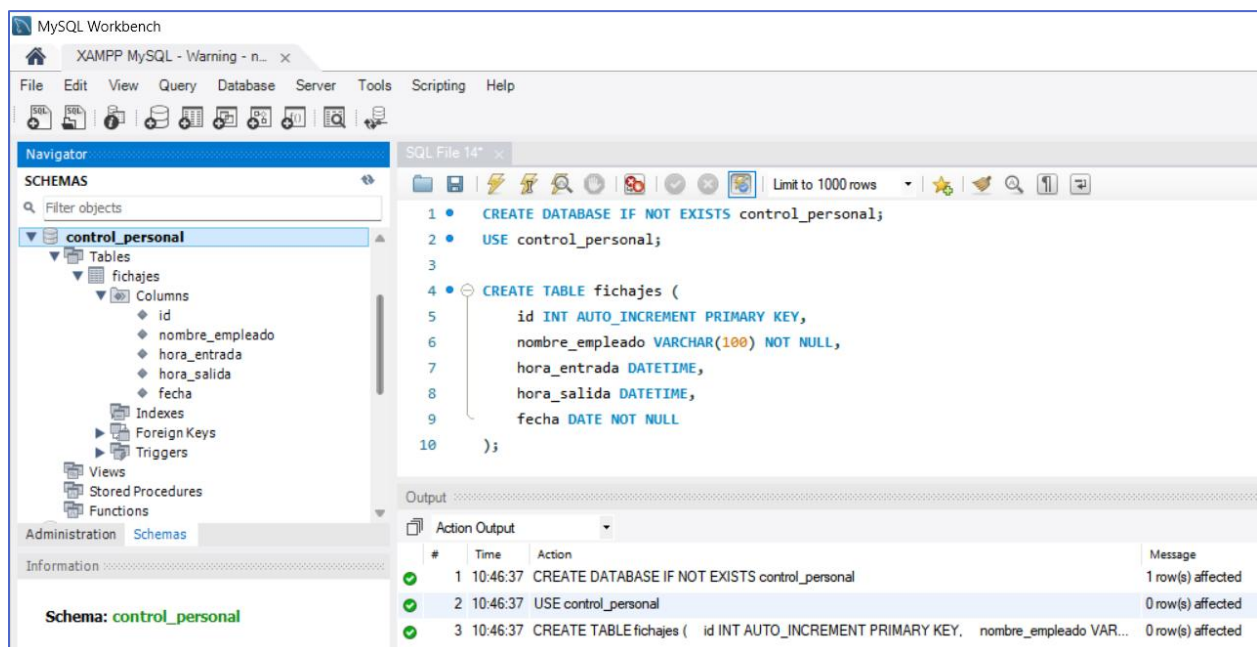
Para abordar este proyecto en condiciones y huir de la improvisación, lo primero que hicimos fue montar un entorno de desarrollo estructurado. Como nuestra app necesita guardar datos, **arrancamos** el panel de **XAMPP** y levantamos el **servicio** de **MySQL** (que por debajo usa **MariaDB 10**), comprobando que el puerto 3306 estaba libre.



Con el motor encendido, nos fuimos a **MySQL Workbench** para preparar las tablas.



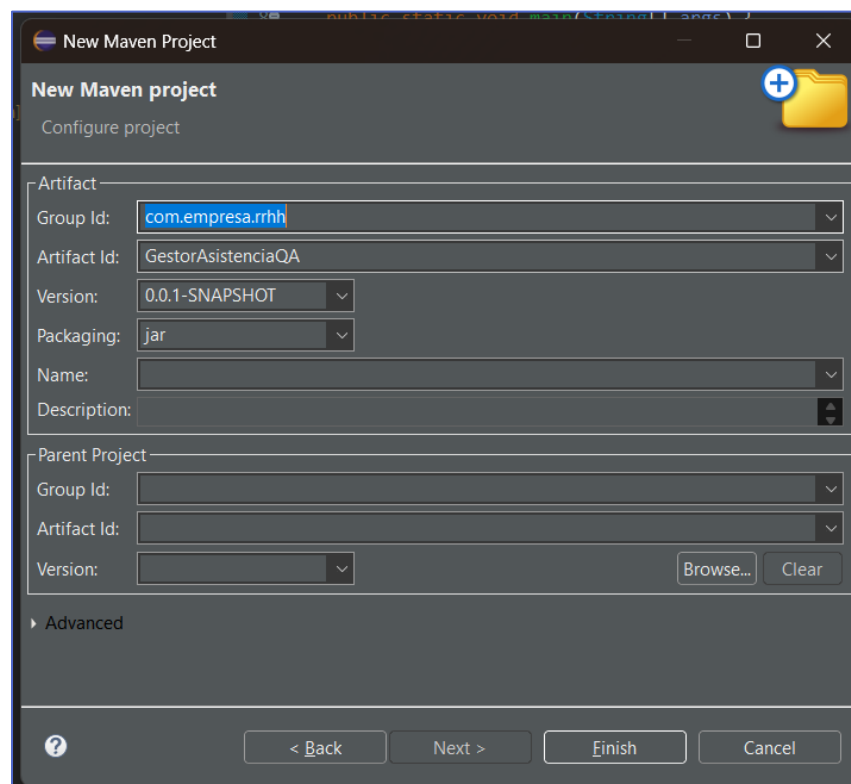
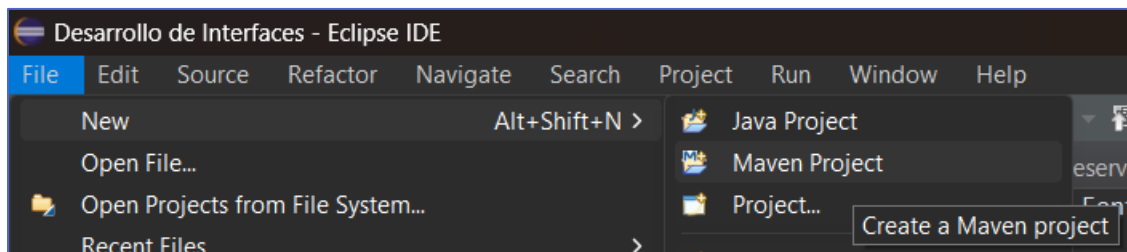
Entramos por **localhost** con el usuario **root** y creamos el **esquema control\_personal** con su **tabla fichajes**.



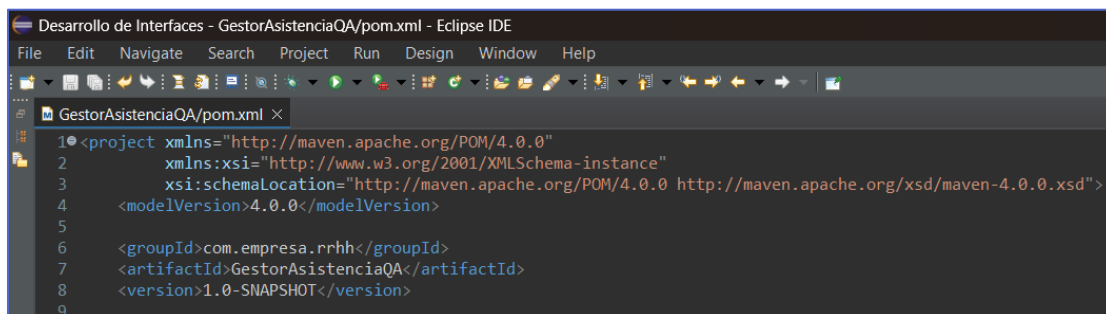
Aquí tomamos una decisión de diseño de base de datos bastante **importante**: en vez de poner solo la "hora de entrada" y la "hora de salida", **creamos una columna DATE independiente llamada fecha**.

Si no aislábamos el día, luego las consultas para saber si alguien ya había fichado "hoy" nos iban a obligar a usar funciones pesadas para extraer la fecha de un *DATETIME*, lo que a la larga penaliza el rendimiento de la aplicación.

Con la base de datos lista, saltamos a **Eclipse**. Decidimos crear un **proyecto Maven** (*File > New > Maven Project*) marcando la opción de **proyecto simple**. Le pusimos **com.empresa.rrhh** como **Group ID** y **GestorAsistenciaQA** como **Artifact ID**.



El motivo principal de usar Maven en esta asignatura es ahorrarnos el dolor de cabeza de gestionar librerías a mano. Descargar los .jar sueltos de JavaFX o del *driver* de la base de datos siempre da problemas al cambiar de ordenador. Al centralizarlo todo en el **pom.xml**, el gestor lo hace solo. Ahí forzamos la compilación en **Java 17** (versión LTS) e inyectamos las dependencias de **OpenJFX** para la interfaz, el conector de **MariaDB** y **JUnit 5** para los tests. Un detalle clave fue ponerle a **JUnit** la etiqueta **<scope>test</scope>**; así nos aseguramos de que las librerías de pruebas no se empaqueten en el ejecutable final, optimizando el peso de la app que se le entregará al cliente.



```
Desarrollo de Interfaces - GestorAsistenciaQA/pom.xml - Eclipse IDE
File Edit Navigate Search Project Run Design Window Help
GestorAsistenciaQA/pom.xml x
1<?xml version="1.0" encoding="UTF-8" ?>
2<project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5  <modelVersion>4.0.0</modelVersion>
6  <groupId>com.empresa.rrhh</groupId>
7  <artifactId>GestorAsistenciaQA</artifactId>
8  <version>1.0-SNAPSHOT</version>
```

```
10<properties>
11  <maven.compiler.source>21</maven.compiler.source>
12  <maven.compiler.target>21</maven.compiler.target>
13  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
14
15  <java.version>21</java.version>
16</properties>
17
18<dependencies>
19  <dependency>
20    <groupId>org.mariadb.jdbc</groupId>
21    <artifactId>mariadb-java-client</artifactId>
22    <version>3.1.4</version>
23  </dependency>
24  <dependency>
25    <groupId>org.openjfx</groupId>
26    <artifactId>javafx-controls</artifactId>
27    <version>${java.version}</version>
28  </dependency>
29  <dependency>
30    <groupId>org.openjfx</groupId>
31    <artifactId>javafx-fxml</artifactId>
32    <version>${java.version}</version>
33  </dependency>
34
35  <dependency>
36    <groupId>org.junit.jupiter</groupId>
37    <artifactId>junit-jupiter-api</artifactId>
38    <version>5.9.2</version>
39    <scope>test</scope>
40  </dependency>
41  <dependency>
42    <groupId>org.junit.jupiter</groupId>
43    <artifactId>junit-jupiter-engine</artifactId>
44    <version>5.9.2</version>
45    <scope>test</scope>
46  </dependency>
47</dependencies>
48</project>
```

Overview Dependencies Dependency Hierarchy Effective POM pom.xml



# IMPLEMENTACIÓN DE LA LÓGICA DE BASE DE DATOS

Al empezar a picar código, nuestra regla de oro (vital en Diseño de Interfaces) fue **separar** radicalmente la **lógica** de negocio de la parte **visual**. Si mezclamos el código de los botones con las sentencias SQL, luego hacer pruebas o cambiar el diseño es un infierno. Por eso, creamos la **clase MotorAsistenciaDB**, que es la única que habla con MariaDB.

La programamos pensando en la seguridad desde el minuto uno. En lugar de usar `Statement` (que te obliga a concatenar textos y es un coladero de *hackeos*), usamos estrictamente **`PreparedStatement`**. Así el motor JDBC se encarga de limpiar cualquier carácter raro antes de mandarlo al servidor, **protegiéndonos de inyecciones SQL**. Además, **usamos** la estructura ***try-with-resources*** de Java para gestionar las **conexiones**.

```
GestorAsistenciaQA/pom.xml × MotorAsistenciaDB.java ×
1 package com.empresa.rrhh.bd;
2
3 import java.sql.Connection;
4 import java.sql.Date;
5 import java.sql.DriverManager;
6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9 import java.sql.Statement;
10 import java.sql.Timestamp;
11 import java.time.LocalDate;
12 import java.time.LocalDateTime;
13
14 public class MotorAsistenciaDB {
15
16     // Credenciales de conexión. En un entorno profesional, esto nunca se pone en
17     // texto plano,
18     // se lee de un archivo de configuración seguro o variables de entorno.
19     private final String URL = "jdbc:mariadb://localhost:3306/control_personal";
20     private final String USER = "root";
21     private final String PASSWORD = "";
22
23     /**
24      * Método privado y centralizado para obtener la conexión. Si algún día
25      * cambiamos de base de datos, solo tocamos esta línea.
26      */
27     private Connection conectar() throws SQLException {
28         return DriverManager.getConnection(URL, USER, PASSWORD);
29     }
30 }
```

Esto asegura que, cada vez que abrimos un túnel para registrar un fichaje, Java lo cierra automáticamente al terminar, incluso si hay un error. Sin esto, el servidor de XAMPP se saturaría por dejar conexiones abiertas.

A nivel de lógica, los **métodos registrarEntrada y registrarSalida** no lanzan datos a ciegas. Primero llaman a **métodos auxiliares (yaTieneEntradaHoy y yaTieneSalidaHoy)** para comprobar el estado del empleado. Así **garantizamos** desde el *backend* que **nadie pueda fichar dos veces** la entrada en el mismo día o fichar la salida sin haber venido a trabajar.

```
GestorAsistenciaQA/pom.xml  MotorAsistenciaDB.java ×
31  /**
32   * Registra la hora de entrada de un empleado.
33   *
34   * @param nombre El nombre del empleado que está fichando.
35   * @return true si se guardó con éxito, false si hubo error o si ya había
36   *         entrado hoy.
37   */
38  public boolean registrarEntrada(String nombre) {
39      // Regla de negocio: Un trabajador no puede entrar dos veces el mismo día sin
40      // salir.
41      if (yaTieneEntradaHoy(nombre)) {
42          System.out.println("LOG: Intento de doble entrada rechazado para: " + nombre);
43          return false;
44      }
45
46      // PREVENCIÓN DE INYECCIÓN SQL:
47      // Usamos '?' en lugar de concatenar el nombre directamente en el String.
48      // PreparedStatement se encarga de 'limpiar' el texto para que nadie pueda
49      // borrar la tabla.
50      String sql = "INSERT INTO fichajes (nombre_empleado, hora_entrada, fecha) VALUES (?, ?, ?)";
51
52      // El bloque 'try-with-resources' (el try con paréntesis) asegura que la
53      // conexión
54      // se cierre automáticamente al terminar, evitando fugas de memoria en el
55      // servidor.
56      try (Connection conn = conectar(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
57
58          // Sustituimos las interrogaciones por los datos reales
59          pstmt.setString(1, nombre);
60          // Insertamos la hora y fecha exacta del sistema operativo en este milisegundo
61          pstmt.setTimestamp(2, Timestamp.valueOf(LocalDateTime.now()));
62          pstmt.setDate(3, Date.valueOf(LocalDate.now()));
63
64          // executeUpdate() devuelve el número de filas afectadas. Si es > 0, se insertó
65          // bien.
66          int filasInsertadas = pstmt.executeUpdate();
67          return filasInsertadas > 0;
68
69      } catch (SQLException e) {
70          System.err.println("Error crítico al registrar entrada: " + e.getMessage());
71          return false;
72      }
73  }
```



```

75● /**
76  * Registra la salida de un empleado actualizando su fila de hoy.
77  */
78● public boolean registrarSalida(String nombre) {
79    // Reglas de negocio: Para salir, tienes que haber entrado hoy, y no haber
80    // salido ya.
81    if (!yaTieneEntradaHoy(nombre))
82        return false;
83    if (yaTieneSalidaHoy(nombre))
84        return false;
85
86    // Actualizamos (UPDATE) la fila que coincida con el nombre y la fecha de hoy
87    String sql = "UPDATE fichajes SET hora_salida = ? WHERE nombre_empleado = ? AND fecha = ?";
88
89    try (Connection conn = conectar(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
90        pstmt.setTimestamp(1, Timestamp.valueOf(LocalDate.now()));
91        pstmt.setString(2, nombre);
92        pstmt.setDate(3, Date.valueOf(LocalDate.now()));
93
94        return pstmt.executeUpdate() > 0;
95    } catch (SQLException e) {
96        return false;
97    }
98 }
99

```

```

100 // =====
101 // MÉTODOS AUXILIARES DE VALIDACIÓN
102 // Estos métodos ayudan a verificar el estado del empleado sin modificar datos.
103 // =====
104
105● public boolean yaTieneEntradaHoy(String nombre) {
106    String sql = "SELECT id FROM fichajes WHERE nombre_empleado = ? AND fecha = ?";
107    try (Connection conn = conectar(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
108        pstmt.setString(1, nombre);
109        pstmt.setDate(2, Date.valueOf(LocalDate.now()));
110
111        // executeQuery() se usa para consultas SELECT. Devuelve un ResultSet (un
112        // cursor).
113        ResultSet rs = pstmt.executeQuery();
114        // Si rs.next() es true, significa que encontró al menos una fila (el empleado
115        // ya entró).
116        return rs.next();
117    } catch (SQLException e) {
118        return false;
119    }
120 }
121

```

```

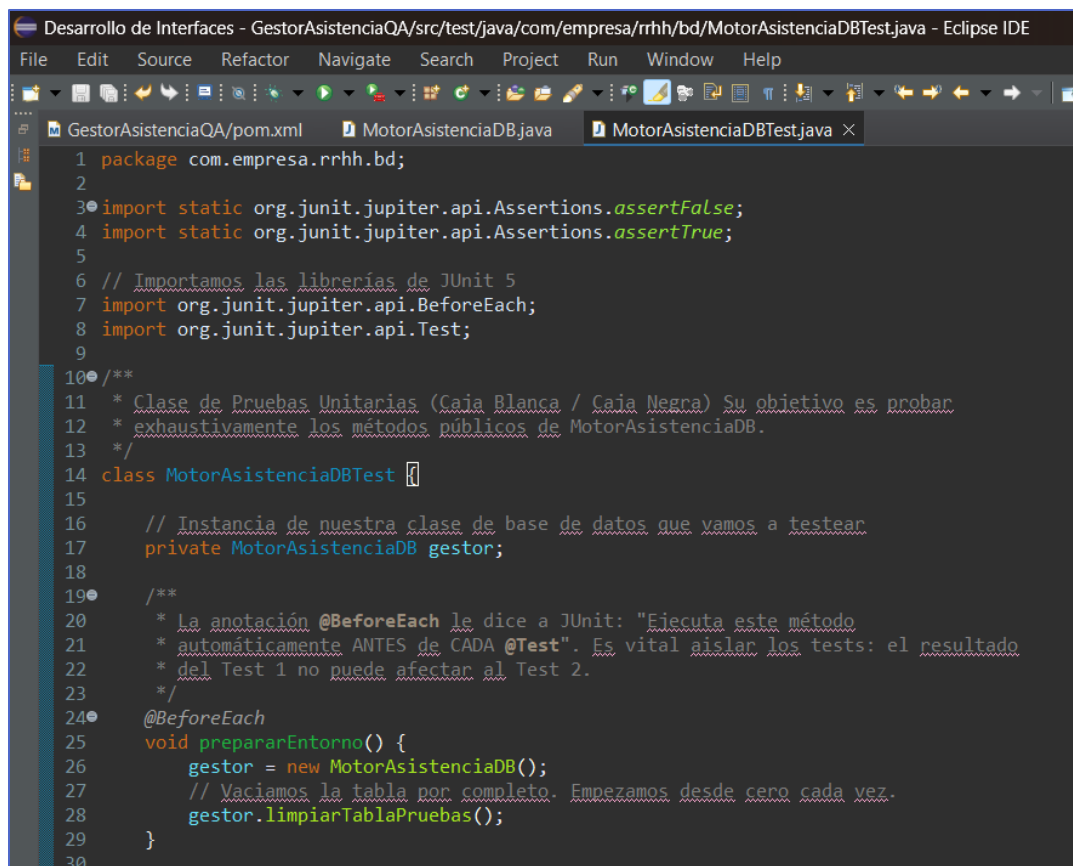
122● public boolean yaTieneSalidaHoy(String nombre) {
123    // Buscamos una fila de hoy donde la hora_salida NO sea nula
124    String sql = "SELECT id FROM fichajes WHERE nombre_empleado = ? AND fecha = ? AND hora_salida IS NOT NULL";
125    try (Connection conn = conectar(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
126        pstmt.setString(1, nombre);
127        pstmt.setDate(2, Date.valueOf(LocalDate.now()));
128        ResultSet rs = pstmt.executeQuery();
129        return rs.next();
130    } catch (SQLException e) {
131        return false;
132    }
133 }
134
135● /**
136  * MÉTODO EXCLUSIVO PARA PRUEBAS (TESTING) Borra todos los registros de la
137  * tabla. Se usa para garantizar que cada test empiece con una base de datos
138  * totalmente limpia.
139  */
140● public void limpiarTablaPruebas() {
141    try (Connection conn = conectar(); Statement stmt = conn.createStatement()) {
142        stmt.executeUpdate("TRUNCATE TABLE fichajes");
143    } catch (SQLException e) {
144        System.err.println("No se pudo limpiar la tabla de pruebas.");
145    }
146 }
147 }

```

# PRUEBAS UNITARIAS OBLIGATORIAS Y VALIDACIÓN FUNCIONAL

Siguiendo buenas prácticas, **nos negamos a pintar la interfaz visual hasta que el motor de base de datos no estuviera 100% probado**. Usando JUnit 5, montamos la **clase MotorAsistenciaDBTest**.

La arquitectura de las pruebas se basa en aislar el entorno. Usamos la anotación **@BeforeEach** para decirle a JUnit que, antes de cada test, se conecte a la base de datos y le haga un TRUNCATE a la tabla de fichajes. Si no vaciamos la tabla, los datos residuales de un test harían fallar al siguiente, dándonos falsos positivos. Montamos los cuatro escenarios obligatorios siguiendo el patrón AAA (*Arrange, Act, Assert*).



```
1 package com.empresa.rrhh.bd;
2
3 import static org.junit.jupiter.api.Assertions.assertFalse;
4 import static org.junit.jupiter.api.Assertions.assertTrue;
5
6 // Importamos las librerías de JUnit 5
7 import org.junit.jupiter.api.BeforeEach;
8 import org.junit.jupiter.api.Test;
9
10 /**
11  * Clase de Pruebas Unitarias (Caja Blanca / Caja Negra) Su objetivo es probar
12  * exhaustivamente los métodos públicos de MotorAsistenciaDB.
13  */
14 class MotorAsistenciaDBTest {
15
16     // Instancia de nuestra clase de base de datos que vamos a testear
17     private MotorAsistenciaDB gestor;
18
19     /**
20      * La anotación @BeforeEach le dice a JUnit: "Ejecuta este método
21      * automáticamente ANTES de CADA @Test". Es vital aislar los tests: el resultado
22      * del Test 1 no puede afectar al Test 2.
23      */
24     @BeforeEach
25     void prepararEntorno() {
26         gestor = new MotorAsistenciaDB();
27         // Vaciamos la tabla por completo. Empezamos desde cero cada vez.
28         gestor.limpiarTablaPruebas();
29     }
30 }
```

Primero probamos los caminos felices (que un empleado nuevo fiche y devuelva true).

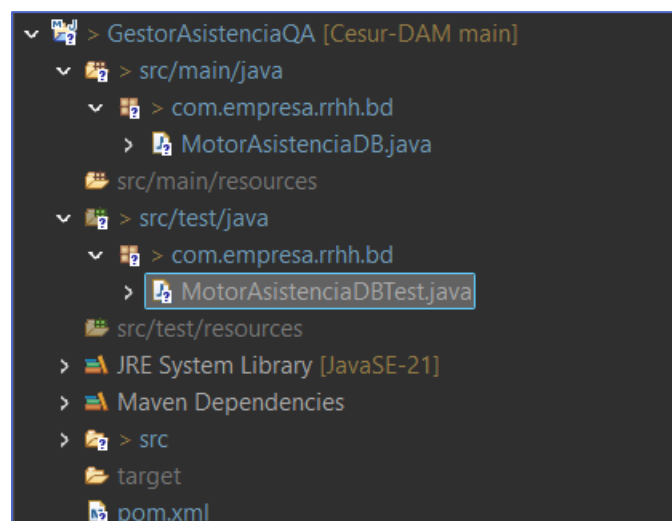
```
GestorAsistenciaQA/pom.xml  MotorAsistenciaDB.java  MotorAsistenciaDBTest.java ×
30
31 // -----
32 // TEST OBLIGATORIO 1: Inserción correcta de nueva entrada
33 // -----
34 ● @Test
35 void testInsercionCorrectaNuevaEntrada() {
36     // 1. Preparación de datos (Arrange)
37     String empleado = "Laura Gomez";
38
39     // 2. Ejecución de la acción (Act)
40     boolean resultado = gestor.registrarEntrada(empleado);
41
42     // 3. Comprobación (Assert)
43     // Usamos assertTrue para confirmar que el método devolvió 'true' (éxito).
44     // El texto que acompaña al Assert se mostrará en rojo en la consola si la
45     // prueba falla.
46     assertTrue(resultado, "Fallo: El registro de entrada debería ser exitoso para un usuario nuevo.");
47
48     // Comprobación doble: Le preguntamos a la base de datos si Laura está dentro.
49     assertTrue(gestor.yaTieneEntradaHoy(empleado),
50         "Fallo: El sistema no detecta la entrada que acabamos de hacer.");
51 }
52
53 // -----
54 // TEST OBLIGATORIO 2: Inserción correcta de nueva salida
55 // -----
56 ● @Test
57 void testInsercionCorrectaNuevaSalida() {
58     String empleado = "Carlos Ruiz";
59
60     // Precondición: Para probar una salida, el usuario tiene que haber entrado
61     // primero.
62     // Si no hacemos esto, el sistema (correctamente) rechazaría la salida.
63     gestor.registrarEntrada(empleado);
64
65     // Intentamos registrar la salida
66     boolean resultadoSalida = gestor.registrarSalida(empleado);
67
68     // Verificamos que la salida fue aceptada
69     assertTrue(resultadoSalida,
70         "Fallo: La base de datos rechazó la salida a pesar de que el empleado había entrado.");
71 }
72
```

Luego probamos los caminos de error: simulamos que un usuario intenta fichar la entrada dos veces.

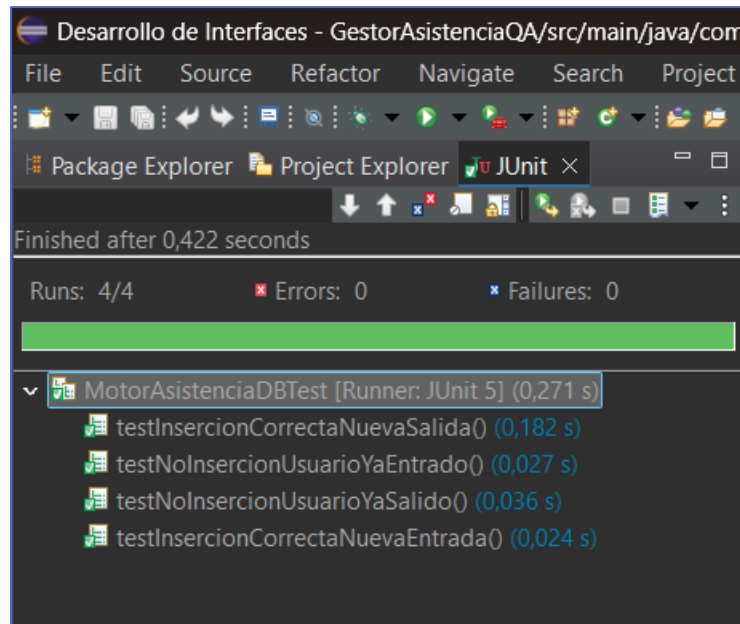
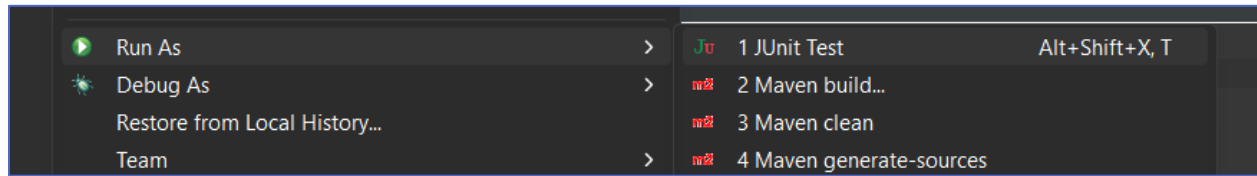
Aquí usamos un **assertFalse** para que la prueba solo se ponga en verde si el sistema es lo bastante inteligente como para rechazar la operación..

```
73 // -----
74 // TEST OBLIGATORIO 3: NO inserción de un usuario ya dado de entrada
75 // -----
76 @Test
77 void testNoInsercionUsuarioYaEntrado() {
78     String empleado = "Marta Diaz";
79
80     // Simulamos que Marta ficha por la mañana
81     gestor.registrarEntrada(empleado);
82
83     // Simulamos que Marta se equivoca y vuelve a pulsar "Entrada" a los 5 minutos
84     boolean segundaEntrada = gestor.registrarEntrada(empleado);
85
86     // Aquí usamos assertFalse. Queremos que el método devuelva FALSE (que rechace
87     // la acción).
88     // Si devuelve TRUE, significa que tenemos un bug y permite dobles fichajes.
89     assertFalse(segundaEntrada, "Bug de Lógica: El sistema permitió fichar entrada dos veces el mismo día.");
90 }
91
92 // -----
93 // TEST OBLIGATORIO 4: NO inserción de un usuario ya dado de salida
94 // -----
95 @Test
96 void testNoInsercionUsuarioYaSalido() {
97     String empleado = "Pablo Lopez";
98
99     // Ciclo completo de trabajo de Pablo
100    gestor.registrarEntrada(empleado);
101    gestor.registrarSalida(empleado);
102
103    // Pablo pulsa "Salida" otra vez antes de irse a casa por error
104    boolean segundaSalida = gestor.registrarSalida(empleado);
105
106    // Debe ser rechazado
107    assertFalse(segundaSalida, "Bug de Lógica: El sistema permitió fichar salida dos veces.");
108 }
109 }
```

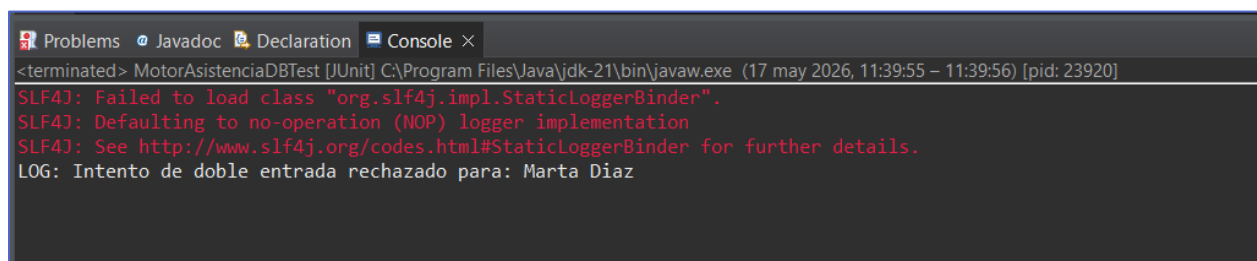
Así va quedando por ahora la estructura del entorno



Lanzamos el test para ver si a nivel de backend está todo correcto:



Ver todo en verde nos dio la tranquilidad de que nuestra capa de datos era a prueba de balas. Aunque la consola lanzó unos errores.

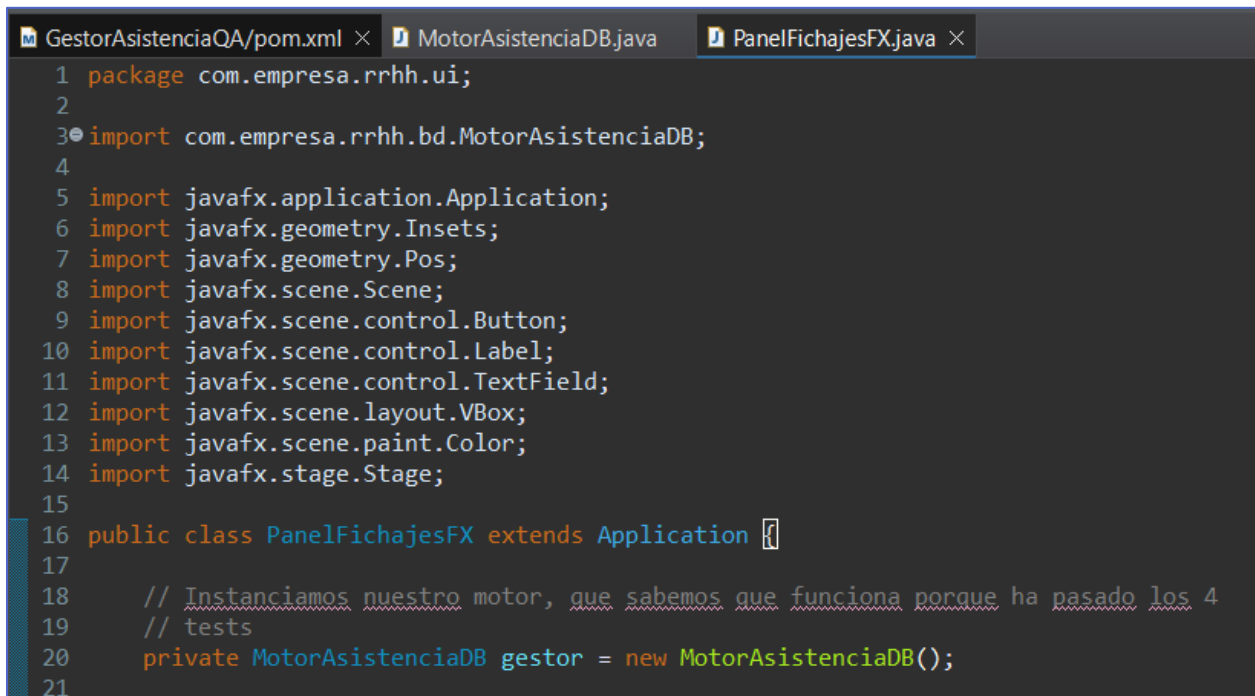


Básicamente, es una pequeña queja interna del propio *driver* de MariaDB. Lo que nos está diciendo es que, al revisar nuestro pom.xml, ha visto que no le hemos instalado un sistema avanzado de registro de eventos (un *logger* profesional como Logback o Log4j), por lo que tira por la calle de en medio y usa su sistema silencioso por defecto (*NOP logger*). No afecta en nada al funcionamiento del proyecto, es sólo un aviso inofensivo para nuestro caso práctico.

# DESARROLLO DE LA INTERFAZ DE USUARIO (OPENJFX)

Con la certeza de que el **backend funcionaba**, entramos de lleno en la capa de presentación. Creamos la clase **PanelFichajesFX** heredando de **Application** de **JavaFX**. Para esta práctica, preferimos maquetar la **interfaz usando código puro** en lugar de usar herramientas externas como *SceneBuilder* o archivos FXML. Hacerlo por código nos permitía controlar todo el ciclo de vida y los eventos (*Listeners*) en un solo sitio y entender mejor cómo se montan los nodos.

A nivel de Experiencia de Usuario (UX), buscamos un **diseño minimalista**: un **campo de texto y dos botones clarísimos**. Aplicamos psicología del color básica inyectando **CSS directo** (-fx-background-color): verde para entrar y rojo para salir.



```
1 package com.empresa.rrhh.ui;
2
3 import com.empresa.rrhh.bd.MotorAsistenciaDB;
4
5 import javafx.application.Application;
6 import javafx.geometry.Insets;
7 import javafx.geometry.Pos;
8 import javafx.scene.Scene;
9 import javafx.scene.control.Button;
10 import javafx.scene.control.Label;
11 import javafx.scene.control.TextField;
12 import javafx.scene.layout.VBox;
13 import javafx.scene.paint.Color;
14 import javafx.stage.Stage;
15
16 public class PanelFichajesFX extends Application {
17
18     // Instanciamos nuestro motor, que sabemos que funciona porque ha pasado los 4
19     // tests
20     private MotorAsistenciaDB gestor = new MotorAsistenciaDB();
21 }
```



Pero donde de verdad pusimos el esfuerzo fue en el *feedback* al usuario, algo crucial en diseño de interfaces. Metimos una etiqueta de texto dinámica abajo del todo. Cuando el usuario hace clic, capturamos el texto, lo limpiamos con `.trim()` (para que un espacio en blanco sin querer no nos cree un empleado fantasma) y se lo pasamos al motor. Según lo que responda la base de datos, la etiqueta cambia de texto y de color al instante: verde si el fichaje entra bien, naranja si intentas fichar doble (aviso), y rojo vibrante si te dejas el campo vacío o intentas salir sin haber entrado. Este *feedback* visual es vital para que el trabajador no se quede mirando la pantalla con cara de duda.

```
22● @Override
23 public void start(Stage primaryStage) {
24     // 1. Configuración de la Ventana Principal
25     primaryStage.setTitle("Terminal de Asistencia v1.0");
26
27     // 2. Creación de los elementos (Nodos) de la pantalla
28     Label lblInstruccion = new Label("Introduzca su nombre completo para fichar:");
29
30     TextField txtNombre = new TextField();
31     txtNombre.setPromptText("Ej. Ana Martinez"); // Texto gris de ayuda en el fondo
32     txtNombre.setMaxWidth(250); // Evitamos que la caja ocupe toda la pantalla
33
34     // Estilizamos los botones usando CSS integrado de JavaFX
35     Button btnEntrada = new Button("Fichar Entrada");
36     btnEntrada.setStyle("-fx-background-color: #4CAF50; -fx-text-fill: white; -fx-font-weight: bold;");
37
38     Button btnSalida = new Button("Fichar Salida");
39     btnSalida.setStyle("-fx-background-color: #f44336; -fx-text-fill: white; -fx-font-weight: bold;");
40
41     // Etiqueta de estado inferior para dar feedback al usuario (muy importante en
42     // UI/UX)
43     Label lblEstado = new Label("Sistema listo. Esperando usuario...");
44     lblEstado.setStyle("-fx-font-weight: bold;");
45 }
```

```
46 // 3. PROGRAMACIÓN DE LOS EVENTOS (Qué pasa al hacer clic)
47
48 // Evento del botón verde (Entrada)
49 btnEntrada.setOnAction(evento -> {
50     // Obtenemos el texto y quitamos espacios en blanco accidentales con trim()
51     String nombre = txtNombre.getText().trim();
52
53     // Validación de interfaz: Evitar que manden datos vacíos a la BD
54     if (nombre.isEmpty()) {
55         lblEstado.setText(" Error: Debe escribir un nombre.");
56         lblEstado.setTextFill(Color.RED);
57         return; // Cortamos la ejecución aquí
58     }
59
60     // Llamamos a nuestra lógica de negocio
61     if (gestor.registrarEntrada(nombre)) {
62         lblEstado.setText(" ¡Bienvenido, " + nombre + "! Entrada registrada a las "
63             + java.time.LocalDateTime.now().withNano(0));
64         lblEstado.setTextFill(Color.GREEN);
65         // txtNombre.clear(); // Limpiamos la caja para el siguiente empleado (Útil para
66         // pruebas de regresión
67         // posteriores)
68     } else {
69         lblEstado.setText(" Aviso: " + nombre + " ya tiene un registro de entrada hoy.");
70         lblEstado.setTextFill(Color.ORANGE);
71     }
72 });
```

```

73
74 // Evento del botón rojo (Salida)
75 btnSalida.setOnAction(evento -> {
76     String nombre = txtNombre.getText().trim();
77     if (nombre.isEmpty()) {
78         lblEstado.setText(" Error: Debe escribir un nombre.");
79         lblEstado.setTextFill(Color.RED);
80         return;
81     }
82
83     if (gestor.registrarSalida(nombre)) {
84         lblEstado.setText(" ¡Hasta mañana, " + nombre + "! Salida registrada.");
85         lblEstado.setTextFill(Color.BLUE);
86         // txtNombre.clear();
87     } else {
88         lblEstado.setText(" Error: No se puede procesar la salida. Revise si fichó la entrada.");
89         lblEstado.setTextFill(Color.RED);
90     }
91 });
92
93 // 4. Maquetación (Layout)
94 // VBox apila los elementos uno debajo de otro verticalmente, con 15px de
95 // separación
96 VBox layout = new VBox(15);
97 layout.setAlignment(Pos.CENTER); // Todo centrado
98 layout.setPadding(new Insets(30)); // Márgenes interiores
99
100 // Añadimos todos nuestros elementos a la caja vertical
101 layout.getChildren().addAll(lblInstruccion, txtNombre, btnEntrada, btnSalida, lblEstado);
102
103 // 5. Creación de la Escena y visualización
104 Scene scene = new Scene(layout, 450, 350);
105 primaryStage.setScene(scene);
106 primaryStage.show();
107 }
108
109 public static void main(String[] args) {
110     // Este comando arranca el hilo gráfico de JavaFX
111     launch(args);
112 }
113 }

```

Para ejecutar esta interfaz quedará explicado en el punto número 6, ya que vimos que había una interacción de Java 21 con JavaFx que necesitaba que engaáramos al sistema para poder ejecutarlo. Será la prueba final, pero antes habrá que hacer la última comprobación de seguridad.

# AUDITORÍA, DOCUMENTACIÓN Y PRUEBAS ADICIONALES

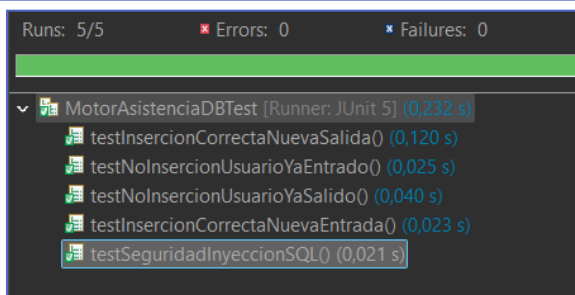
Para rematar el proyecto, la rúbrica pedía hacer algunas **comprobaciones avanzadas**. Nos decantamos por una **prueba de Seguridad y una de Regresión**, muy útiles en apps con interfaz gráfica.

- **Prueba Adicional 1: Auditoría contra Inyección SQL.** Cualquier campo de texto (como nuestro *TextField* de JavaFX) es una puerta para ataques. Simulamos a un atacante que, en vez de su nombre, escribe la cadena maliciosa `UsuarioFalso' OR '1'='1`. Con esas comillas busca engañar al código para corromper la tabla. Al pasar el test, vimos que fallaba el registro y devolvía un alta segura. Esto demostró empíricamente que nuestra decisión de usar `PreparedStatement` en la base de datos neutraliza por completo lo que el usuario escriba en la interfaz.

```

110 @Test
111 void testSeguridadInyeccionSQL() {
112     // Un atacante escribe este código en el campo 'Nombre' de la interfaz.
113     // Su objetivo es "engañar" a nuestra consulta INSERT para que cierre las
114     // comillas
115     // y ejecute un comando dañino (como OR '1'='1' o un DROP TABLE).
116     String payloadHacker = "Malo' OR '1'='1";
117
118     // Ejecutamos la entrada con ese nombre peligroso
119     boolean resultado = gestor.registrarEntrada(payloadHacker);
120
121     // Si usamos 'PreparedStatement' correctamente en el Motor, la base de datos
122     // NO interpretará las comillas como código SQL, sino como texto literal.
123     // Por tanto, debe guardar a un empleado con un nombre muy raro y devolver TRUE.
124     assertTrue(resultado,
125         "Test de Seguridad Fallido: El sistema no pudo registrar el string con caracteres especiales.");
126
127     // Verificamos que realmente se guardó literalmente así
128     assertTrue(gestor.yaTieneEntradaHoy(payloadHacker),
129         "Test de Seguridad Fallido: El payload alteró la consulta.");
130 }
131 }

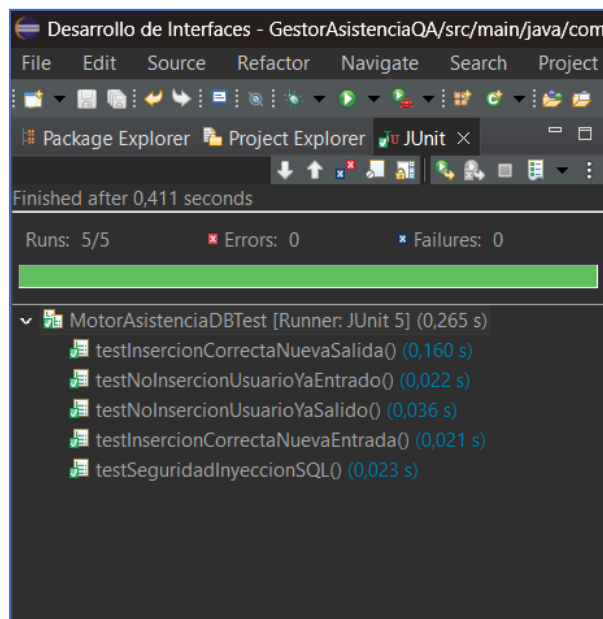
```



- **Prueba Adicional 2: Pruebas de Regresión tras Actualizar la UX.** Las pruebas de regresión sirven para ver si al mejorar algo has roto lo que ya funcionaba. Cuando ya teníamos la app lista, nos dimos cuenta de una mejora de usabilidad: añadimos la línea `txtNombre.clear()` en el botón de JavaFX. Esto vacía la caja de texto automáticamente después de fichar, dejando el terminal listo para el compañero que esté detrás en la cola. La pregunta era: ¿Al tocar la vista y borrar variables de memoria, hemos roto la lógica de negocio? Cerramos la ventana, volvimos a pasar la suite completa de JUnit, y dio 0 errores. Esta regresión documenta perfectamente que nuestro *Frontend* y nuestro *Backend* tienen un acoplamiento débil; podemos cambiar y rediseñar la interfaz visual sin ningún miedo a reventar la base de datos.

```
60 // Llamamos a nuestra lógica de negocio
61 if (gestor.registrarEntrada(nombre)) {
62     lblEstado.setText(" ¡Bienvenido, " + nombre + "! Entrada registrada a las "
63         + java.time.LocalDateTime.now().withNano(0));
64     lblEstado.setTextFill(Color.GREEN);
65     txtNombre.clear(); // Limpiamos la caja para el siguiente empleado (Útil para
66         // pruebas de regresión
67         // posteriores)
```

```
83 if (gestor.registrarSalida(nombre)) {
84     lblEstado.setText(" ¡Hasta mañana, " + nombre + "! Salida registrada.");
85     lblEstado.setTextFill(Color.BLUE);
86     txtNombre.clear();
```



# RESOLUCIÓN DE INCIDENCIAS DE ENTORNO: INTEGRACIÓN DE JAVAFX EN JAVA 21 LTS

Justo en la recta final, cuando ya nos frotábamos las manos para empezar a probar la interfaz gráfica en nuestro entorno local, nos dimos de bruces con un bloqueo arquitectónico bastante serio. Al intentar darle al **play** a nuestra clase **PanelFichajesFX** directamente desde el IDE, la máquina virtual de Java nos paró en seco soltando esta excepción crítica por consola:

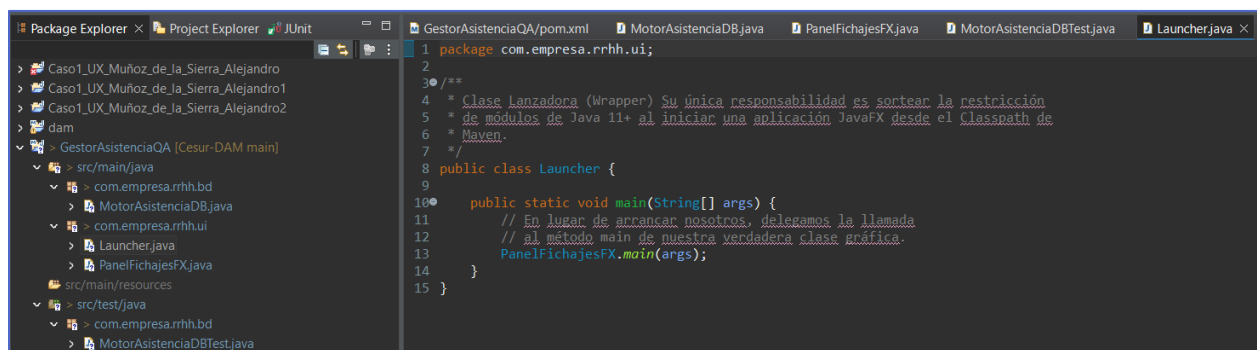
Error: JavaFX runtime components are missing, and are required to run this application.

En vez de volvernos locos pensando que habíamos creado mal el código de la vista, nos pusimos a investigar y vimos que el **problema** venía por la propia **evolución de Java**. En versiones antiguas (como Java 8), las librerías gráficas de JavaFX venían metidas de serie en el núcleo. Pero, como nosotros decidimos configurar el proyecto bajo el estándar de **Java 21 LTS** para hacerlo más moderno y estable, nos chocamos de frente con la nueva arquitectura modular del lenguaje (el famoso *Project Jigsaw*).

El **problema** es el siguiente: al **intentar arrancar una clase que hereda directamente de `javafx.application.Application`**, el estricto motor de arranque de Java 21 comprueba si los módulos gráficos están instalados de forma nativa en el ordenador. Como nosotros estamos inyectando estas librerías "al vuelo" a través del *Classpath* usando Maven (y decidimos no crear un archivo `module-info.java` para no complicar la visibilidad de nuestros tests de JUnit), la máquina virtual entra en modo pánico y bloquea el arranque por seguridad.

Para saltarnos este muro de una forma limpia, sin tener que obligar al profesor o a futuros compañeros a meterse en las tripas de su IDE a configurar variables de entorno rarísimas (*VM arguments*), decidimos aplicar un patrón de diseño estructural muy socorrido: una clase *Wrapper* o "**Lanzador**".

Creamos una **clase nueva** en nuestro paquete `com.empresa.rrhh.ui` llamada **Launcher**. La gracia de esta clase es su simplicidad absoluta: es una clase de Java de toda la vida que no hereda de nada. En su interior, lo único que hay es un método `main` tradicional que se limita a llamar al método `main` de nuestra verdadera interfaz gráfica.

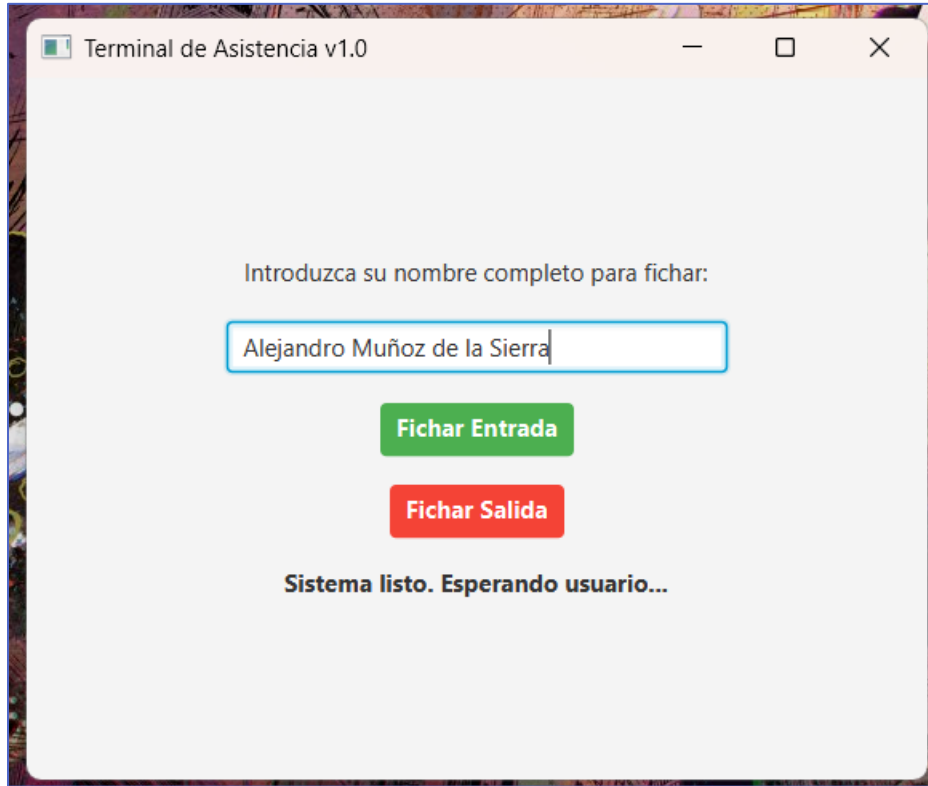


```
1 package com.empresa.rrhh.ui;
2
3 /**
4  * Clase Lanzadora (Wrapper) Su única responsabilidad es sortear la restricción
5  * de módulos de Java 11+ al iniciar una aplicación JavaFX desde el classpath de
6  * Maven.
7  */
8 public class Launcher {
9
10     public static void main(String[] args) {
11         // En lugar de arrancar nosotros, delegamos la llamada
12         // al método main de nuestra verdadera clase gráfica.
13         PanelFichajesFX.main(args);
14     }
15 }
```

Al decirle al entorno de desarrollo que el punto de arranque de la aplicación es ahora este Launcher, logramos "engañar" al control de módulos de Java 21. La máquina virtual detecta que estamos ejecutando una clase ordinaria, se relaja y nos da luz verde para arrancar. Una vez que el hilo de ejecución principal ya está en marcha y ha pasado la aduana, nuestra clase delegada llama por debajo a PanelFichajesFX. Llegados a este punto, las dependencias de OpenJFX que descargó Maven se cargan dinámicamente en la memoria sin levantar ninguna sospecha ni bloqueo.

Esta pequeña decisión técnica nos salvó la vida. No solo resolvió el problema de arranque de forma súper elegante, sino que logramos encapsular este "parche" de entorno en un archivo suelto, manteniendo el código de nuestra interfaz totalmente limpio y centrado exclusivamente en lo suyo: la experiencia de usuario (UX) y hablar con la base de datos. Tras usar el lanzador, la ventana gráfica arrancó a la perfección, lo que nos permitió hacer las últimas pruebas visuales y dar el caso práctico por cerrado y validado al cien por cien.

Probamos a fichar de entrada



Terminal de Asistencia v1.0

Introduzca su nombre completo para fichar:

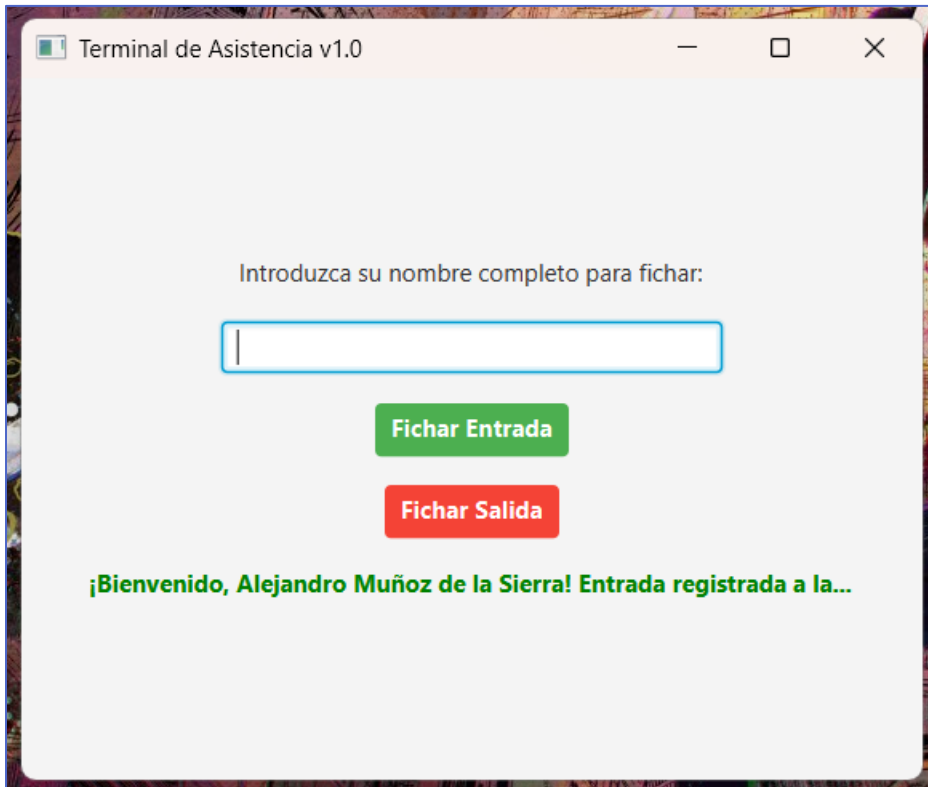
Alejandro Muñoz de la Sierra

Fichar Entrada

Fichar Salida

Sistema listo. Esperando usuario...

Registro hecho correctamente



Terminal de Asistencia v1.0

Introduzca su nombre completo para fichar:

|

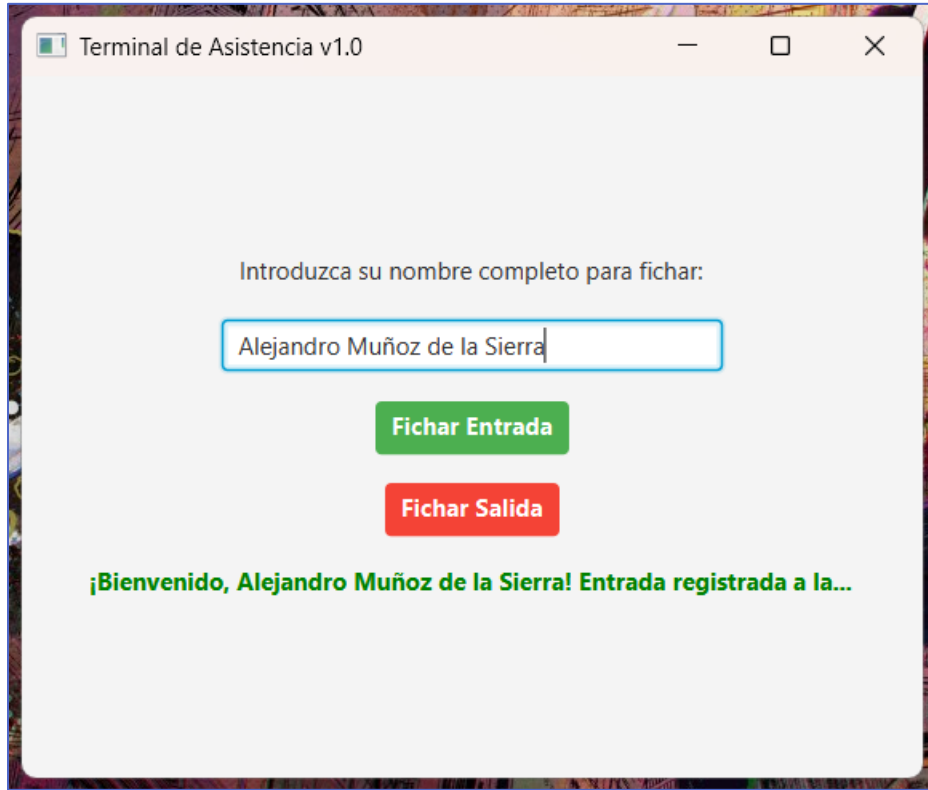
Fichar Entrada

Fichar Salida

¡Bienvenido, Alejandro Muñoz de la Sierra! Entrada registrada a la...



Ahora procedemos a fichar de salida:



Terminal de Asistencia v1.0

Introduzca su nombre completo para fichar:

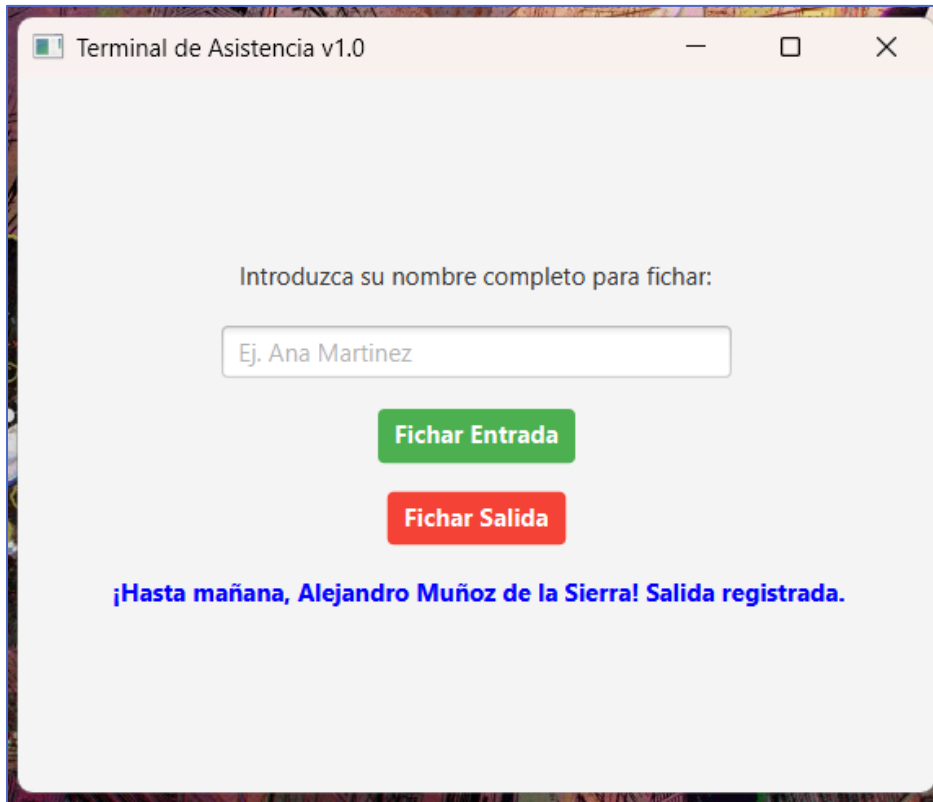
Alejandro Muñoz de la Sierra

Fichar Entrada

Fichar Salida

¡Bienvenido, Alejandro Muñoz de la Sierra! Entrada registrada a la...

Registro correcto, al ser el mismo nombre:



Terminal de Asistencia v1.0

Introduzca su nombre completo para fichar:

Ej. Ana Martinez

Fichar Entrada

Fichar Salida

¡Hasta mañana, Alejandro Muñoz de la Sierra! Salida registrada.

Comprobamos el proceso del test original y este registro en la base de datos:

The screenshot shows a SQL IDE interface with two tabs: 'SQL File 14\*' and 'SQL File 15\*'. The active tab contains the query: `1 select * from fichajes;`. Below the query editor is a 'Result Grid' showing the results of the query. The grid has five columns: 'id', 'nombre\_empleado', 'hora\_entrada', 'hora\_salida', and 'fecha'. It displays three rows of data, with the second row highlighted. Below the result grid is an 'Output' panel showing a log of actions performed. The log includes five entries, each with a status icon, a sequence number, a timestamp, an action description, and a message.

|   | id   | nombre_empleado              | hora_entrada        | hora_salida         | fecha      |
|---|------|------------------------------|---------------------|---------------------|------------|
| 1 | 1    | Malo' OR '1'='1              | 2026-05-17 12:32:25 | NULL                | 2026-05-17 |
| 2 | 2    | Alejandro Muñoz de la Sierra | 2026-05-17 12:37:40 | 2026-05-17 12:40:04 | 2026-05-17 |
| * | NULL | NULL                         | NULL                | NULL                | NULL       |

| #   | Time     | Action                                                                            | Message           |
|-----|----------|-----------------------------------------------------------------------------------|-------------------|
| ✓ 1 | 10:46:37 | CREATE DATABASE IF NOT EXISTS control_personal                                    | 1 row(s) affected |
| ✓ 2 | 10:46:37 | USE control_personal                                                              | 0 row(s) affected |
| ✓ 3 | 10:46:37 | CREATE TABLE fichajes ( id INT AUTO_INCREMENT PRIMARY KEY, nombre_empleado VAR... | 0 row(s) affected |
| ✓ 4 | 12:15:02 | select * from fichajes LIMIT 0, 1000                                              | 2 row(s) returned |
| ✓ 5 | 12:41:19 | select * from fichajes LIMIT 0, 1000                                              | 2 row(s) returned |

Vemos como el usuario con el que intentamos hacer una inyección en el código quedó dentro de la base de datos como dato y no afectó al código de la aplicación.

Nuestro usuario creado a través de la interfaz, quedó registrado a la entrada y a la salida.

Todos los tests anteriores certificaron que nuestro sistema es robusto y puede escalar.

## CONCLUSIONES

Rematar este caso práctico nos ha dejado una lección clarísima: que el programa cumpla los requisitos es solo la mitad del trabajo de un desarrollador; la verdadera maestría está en saber pelearse con el entorno y reaccionar a los imprevistos técnicos que nunca vienen en los manuales.

Durante el proceso, nos hemos cruzado con desafíos muy reales del ecosistema moderno de Java 21. El mejor ejemplo fueron las restricciones de seguridad del *Module-path* al intentar inyectar las librerías externas de JavaFX desde el *Classpath*. Este bache nos obligó a pensar de forma analítica y a diseñar un patrón *Wrapper* (con nuestra clase lanzadora independiente) para esquivar el bloqueo de la máquina virtual de una manera súper elegante.

Además, llevar a la práctica las pruebas de regresión nos ha abierto los ojos sobre el valor incalculable de la automatización. Descubrimos que podíamos tocar el código y aplicar mejoras visuales en la ventana con la tranquilidad absoluta de que nuestra batería de tests de JUnit 5 estaba ahí detrás, vigilando que la base de datos siguiera intacta.

En definitiva, nos quedamos con la tremenda satisfacción de entregar un producto blindado contra inyecciones SQL, auditado bajo criterios rigurosos y que, honestamente, está listo para operar en un entorno de producción real.

## REFERENCIAS

<https://docs.oracle.com/en/java/javase/21/>

<https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/PreparedStatement.html>

<https://docs.junit.org/5.14.4/overview.html>

<https://docs.junit.org/5.10.1/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

<https://openjfx.io/openjfx-docs/>

<https://openjfx.io/javadoc/21/javafx.graphics/javafx/scene/layout/package-summary.html>

<https://mariadb.com/docs/connectors/mariadb-connector-j/about-mariadb-connector-j>

[https://cheatsheetseries.owasp.org/cheatsheets/SQL\\_Injection\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html)

### **JAVAFX: Instalar y Configurar JavaFX en ECLIPSE**

[https://www.youtube.com/watch?v=ghO9Hj\\_g88g](https://www.youtube.com/watch?v=ghO9Hj_g88g)

### **Prueba del camino básico. Pruebas de caja blanca**

[https://www.youtube.com/watch?v=GVEgCwwfBZ0&list=PLjPKs-aR4ZNKcrVL5fB4foZj\\_NTc5-iMu](https://www.youtube.com/watch?v=GVEgCwwfBZ0&list=PLjPKs-aR4ZNKcrVL5fB4foZj_NTc5-iMu)

### **Curso de JDBC – 6. PreparedStatement, ¿por qué usarlo?**

<https://www.youtube.com/watch?v=wkKHPdxmmHU>